ASVS Compliance & Maturity Analyzer

Application Security Verification Standard Report

OWASP ASVS 5.0 · Automated Security Assessment · 18 August 2026 00:30

Project	My Project
Standard	OWASP Application Security Verification Standard 5.0
Tool	ASVS Compliance & Maturity Analyzer
Overall Coverage	**11%**
Maturity Level	**Below Level 1**

Executive Summary

11	103	11%	Below Level 1	7/11
Controls Found	Total Requirements	Coverage	Maturity Level	Categories Active

This automated ASVS 5.0 security assessment analyzed the provided source code and detected 11 implemented security controls out of 103 mapped requirements, achieving 11% overall coverage. Strongest areas include Configuration and Authentication. Priority remediation: Session Management and Cryptography at Rest show lowest coverage. Detailed findings with specific remediation guidance follow.

AI Security Assessment

Security Analysis — ASVS Compliance & Maturity Analyzer

AI Score: 11/100

The codebase shows 11 ASVS 5.0 controls across 7 of 11 categories. Strong implementations in Configuration, Authentication. Semantic analysis identifies 11 priority remediation areas. JWT token expiry, secrets management, and certificate verification require attention.

Strengths Detected

› Configuration: X-Content-Type-Options header detected

› Authentication: Password hashing function detected

› Communication Security: HTTPS/TLS usage detected

› Error Handling and Logging: Exception handling detected

Security Risks

› Verify that all components are up to date.

› Verify that user set passwords are at least 12 characters in length.

› Verify that only strong cipher suites are enabled.

› Verify that the application does not log credentials or payment details.

Recommendations

› Verify that all components are up to date.

› Verify that user set passwords are at least 12 characters in length.

› Verify that only strong cipher suites are enabled.

› Verify that the application does not log credentials or payment details.

Category Breakdown

Security Category	Found	Total	Coverage	Maturity	Top Finding / Gap
Authentication	3	17	**18%**	**Below L1**	Password hashing function detected
Session Management	0	11	**0%**	**Below L1**	Gap: Verify the application never reveals session tok...
Access Control	1	8	**12%**	**Below L1**	CSRF protection detected
Input Validation	1	13	**8%**	**Below L1**	SSRF protection detected
Cryptography at Rest	0	10	**0%**	**Below L1**	Gap: Verify that regulated private data is stored enc...
Error Handling and Logging	1	7	**14%**	**Below L1**	Exception handling detected
Communication Security	1	6	**17%**	**Below L1**	HTTPS/TLS usage detected
Data Protection	0	7	**0%**	**Below L1**	Gap: Verify the application protects sensitive data f...
API and Web Service	1	7	**14%**	**Below L1**	CSRF protection detected
Configuration	3	11	**27%**	**Below L1**	X-Content-Type-Options header detected
Files and Resources	0	6	**0%**	**Below L1**	Gap: Verify that the application will not accept larg...
TOTAL	**11**	**103**	**11%**		

Detailed Findings per Category

Authentication 3/17 (18%)

ID	L	Verification Requirement	Status	Finding / Remediation
2.1.1	1	Verify that user set passwords are at least 12 characters in length. [CWE-521]	Fail	Add length check: if len(password) < 12: raise ValidationError
2.1.2	1	Verify that passwords of at least 64 characters are permitted. [CWE-521]	Fail	Remove any max password length restriction from registration form
2.1.7	1	Verify that passwords are checked against a set of breached passwords. [CWE-521]	Fail	Call pwnedpasswords API: GET https://api.pwnedpasswords.com/range/{hash}
2.1.8	1	Verify that a password strength meter is provided. [CWE-521]	Fail	Add zxcvbn.js to registration form — show strength meter to user
2.2.1	1	Verify that anti-automation controls are effective at mitigating brute force attacks. [CWE-307]	Fail	Install Flask-Limiter: @limiter.limit('5 per minute') on /login
2.2.2	1	Verify that weak authenticators such as SMS are limited to secondary verification. [CWE-304]	Fail	Replace SMS OTP with speakeasy TOTP authenticator app integration
2.3.1	1	Verify that system generated initial passwords are at least 6 characters and expire quickly. [CWE-330]	Fail	Generate temp passwords: secrets.token_hex(8), expire after 24 hours
2.4.1	2	Verify that passwords are stored using an approved one-way key derivation or password hashing function. [CWE-916]	Pass	Line 1167: def pbkdf2(password, salt, iterations, dklen=0, digest=None):
2.4.2	2	Verify that the salt is at least 32 bits in length. [CWE-916]	Fail	bcrypt auto-generates unique 32-bit salt — ensure not using custom salt
2.4.3	2	Verify that if PBKDF2 is used, the iteration count is at least 100,000. [CWE-916]	Fail	Set PBKDF2 iterations: hashlib.pbkdf2_hmac('sha256', pwd, salt, 100000)
2.4.4	2	Verify that if bcrypt is used, the work factor is a minimum of 10. [CWE-916]	Pass	Line 1242: # between the work factor of the current encoded password and the defa
2.5.1	1	Verify that a system generated recovery secret is not sent in clear text. [CWE-640]	Fail	Send reset link via HTTPS: https://app.com/reset?token=<secure_token>
2.5.2	1	Verify password hints or knowledge-based authentication are not present. [CWE-640]	Fail	Delete all security question fields from database and registration form
2.5.4	1	Verify shared or default accounts are not present. [CWE-640]	Pass	Line 1230: # Run the default password hasher once to reduce the timing difference
2.7.1	1	Verify that clear text out of band authenticators such as SMS are not offered by default. [CWE-287]	Fail	Replace SMS: use pyotp.TOTP, generate QR code with qrcode library
2.10.1	2	Verify that integration secrets do not rely on unchanging passwords. [CWE-287]	Fail	Replace static API keys with JWT: jwt.encode(payload, key, algorithm='RS256')
2.10.4	3	Verify API keys are managed securely and not included in the source code. [CWE-798]	Fail	Move secrets to .env: load_dotenv(); key = os.getenv('SECRET_KEY')

Session Management 0/11 (0%)

ID	L	Verification Requirement	Status	Finding / Remediation

3.1.1	1	Verify the application never reveals session tokens in URL parameters or error messages. [CWE-598]	Fail	Never append session token to URL — use Set-Cookie header only
3.2.1	1	Verify the application generates a new session token on user authentication. [CWE-384]	Fail	After login: session.regenerate() or session.clear(); session['user']=id
3.2.2	1	Verify that session tokens possess at least 64 bits of entropy. [CWE-331]	Fail	Use Flask session with SECRET_KEY of 32+ random bytes from os.urandom
3.3.1	1	Verify that logout and expiration invalidate the session token. [CWE-613]	Fail	On logout: session.clear(); response.delete_cookie('session')
3.3.2	1	Verify that re-authentication occurs periodically. [CWE-613]	Fail	Set PERMANENT_SESSION_LIFETIME = timedelta(hours=8) in Flask config
3.4.1	1	Verify that cookie-based session tokens have the Secure attribute set. [CWE-614]	Fail	Set-Cookie: session=X; Secure — add secure=True to all cookie calls
3.4.2	1	Verify that cookie-based session tokens have the HttpOnly attribute set. [CWE-1004]	Fail	Set-Cookie: session=X; HttpOnly — add httponly=True to all cookies
3.4.3	1	Verify that cookie-based session tokens utilize the SameSite attribute. [CWE-16]	Fail	Set-Cookie: session=X; SameSite=Strict — add samesite='Strict'
3.5.2	2	Verify the application uses only stateless session tokens. [CWE-345]	Fail	Replace server sessions with: jwt.encode(payload, key, algorithm='RS256')
3.5.3	2	Verify that stateless session tokens use digital signatures and encryption. [CWE-345]	Fail	Use RS256: jwt.encode(payload, private_key, algorithm='RS256') only
3.7.1	1	Verify the application ensures a full valid login session before allowing sensitive transactions. [CWE-778]	Fail	Add password confirmation step before password change and account deletion

Access Control 1/8 (12%)

ID	L	Verification Requirement	Status	Finding / Remediation
4.1.1	1	Verify that the application enforces access control rules on a trusted service layer. [CWE-602]	Fail	Move all @login_required and permission checks to Flask route decorators
4.1.2	1	Verify that all user and data attributes used by access controls cannot be manipulated by end users. [CWE-639]	Fail	Validate user role claims from JWT before using in access decisions
4.1.3	1	Verify that the principle of least privilege exists. [CWE-285]	Fail	Add check: if resource.user_id != current_user.id: return 403
4.1.5	1	Verify that access controls fail securely including when an exception occurs. [CWE-285]	Fail	Return generic 403 Forbidden — never expose internal permission logic
4.2.1	1	Verify that sensitive data and APIs are protected against IDOR attacks. [CWE-639]	Fail	Check ownership: if db.get(id).owner != user.id: abort(403)
4.2.2	1	Verify that the application enforces a strong anti-CSRF mechanism. [CWE-352]	Pass	Line 621: logger = logging.getLogger("django.security.csrf")
4.3.1	1	Verify administrative interfaces use appropriate multi-factor authentication. [CWE-419]	Fail	Add @require_mfa decorator to all /admin and sensitive routes
4.3.2	1	Verify that directory browsing is disabled unless deliberately desired. [CWE-22]	Fail	Add to nginx: autoindex off; or Apache: Options -Indexes

Input Validation 1/13 (8%)

ID	L	Verification Requirement	Status	Finding / Remediation
5.1.1	1	Verify that the application has defenses against HTTP parameter pollution attacks. [CWE-235]	Fail	Check for duplicate params: if len(set(request.args.keys())) != len(list(request
5.1.3	1	Verify that all input is validated using positive validation (allow lists). [CWE-20]	Fail	Use Marshmallow schema with strict field definitions and unknown=EXCLUDE
5.1.4	1	Verify that structured data is strongly typed and validated against a defined schema. [CWE-20]	Fail	Validate with jsonschema: jsonschema.validate(instance=data, schema=schema)
5.1.5	1	Verify that URL redirects only allow destinations which appear on an allow list. [CWE-601]	Fail	Check redirect: if redirect_url not in ALLOWED_URLS: abort(400)
5.2.1	1	Verify that all untrusted HTML input is properly sanitized. [CWE-79]	Fail	Install bleach: clean_html = bleach.clean(user_input, tags=ALLOWED_TAGS)
5.2.4	1	Verify that the application avoids the use of eval() or other dynamic code execution features. [CWE-95]	Fail	Replace all eval() with json.loads() or ast.literal_eval()
5.2.6	1	Verify that the application protects against SSRF attacks. [CWE-918]	Pass	Line 607: from urllib.parse import urlsplit
5.3.3	1	Verify that output escaping protects against reflected, stored, and DOM based XSS. [CWE-79]	Fail	Enable Jinja2 autoescaping: render_template() with autoescape=True
5.3.4	1	Verify that database queries use parameterized queries or ORMs. [CWE-89]	Fail	Use SQLAlchemy ORM: db.session.execute(text('SELECT * WHERE id=:id'), {'id':id})
5.3.8	1	Verify that the application protects against OS command injection. [CWE-78]	Fail	Replace os.system(cmd) with subprocess.run(['cmd','arg'], shell=False)
5.5.1	1	Verify that serialized objects use integrity checks. [CWE-502]	Fail	Sign data: hmac.new(key, data, sha256).hexdigest() — verify before use
5.5.3	1	Verify that deserialization of untrusted data is avoided. [CWE-502]	Fail	Replace pickle.loads() with json.loads() for all user-supplied data
5.5.4	1	Verify that JSON.parse is used instead of eval() to parse JSON. [CWE-502]	Fail	Replace eval(jsonStr) with JSON.parse(jsonStr) in all JavaScript

Cryptography at Rest 0/10 (0%)

ID	L	Verification Requirement	Status	Finding / Remediation
6.1.1	2	Verify that regulated private data is stored encrypted while at rest, such as PII. [CWE-311]	Fail	Encrypt PII: encrypted = Fernet(key).encrypt(pii_value.encode())
6.1.2	2	Verify that regulated health data is stored encrypted while at rest. [CWE-311]	Fail	Apply AES-256-GCM to all health data fields before database insert
6.2.1	1	Verify that all cryptographic modules fail securely. [CWE-310]	Fail	Wrap crypto in try/except — return generic error never padding details
6.2.2	2	Verify that industry proven cryptographic algorithms are used, not custom coded cryptography. [CWE-327]	Fail	Use AES-256-GCM only: cipher = Cipher(algorithms.AES(key), modes.GCM(iv))
6.2.5	2	Verify that known insecure algorithms such as MD5 and SHA1 are not used. [CWE-326]	Fail	Replace hashlib.md5/sha1 with hashlib.sha256 or bcrypt for passwords
6.2.6	2	Verify that nonces and initialization vectors are not reused. [CWE-326]	Fail	Generate new IV per operation: iv = os.urandom(12) — never reuse IV

6.3.1	2	Verify that all random numbers are generated using approved CSPRNG. [CWE-338]	Fail	Replace random.random() with secrets.token_hex(32) for all tokens
6.3.2	2	Verify that random GUIDs are created using the GUID v4 algorithm and CSPRNG. [CWE-338]	Fail	Replace uuid.uuid1() with uuid.uuid4() for all GUID generation
6.4.1	2	Verify that a secrets management solution such as a key vault is used. [CWE-798]	Fail	Store keys in Vault: client.secrets.kv.v2.read_secret(path='keys/app')
6.4.2	2	Verify that key material is not exposed to the application. [CWE-798]	Fail	Use Vault transit: vault.write('transit/encrypt/key', plaintext=base64(data))

Error Handling and Logging 1/7 (14%)

ID	L	Verification Requirement	Status	Finding / Remediation
7.1.1	1	Verify that the application does not log credentials or payment details. [CWE-532]	Fail	Add log filter: logging.Filter that replaces password= with password=***
7.1.2	1	Verify that the application does not log other sensitive data. [CWE-532]	Fail	Mask PII in logs: email[:3]+'***@'+email.split('@')[1] before logging
7.1.3	2	Verify that the application logs security relevant events. [CWE-778]	Fail	Log auth: app.logger.info('LOGIN user=%s ip=%s result=%s', user, ip, result)
7.3.1	2	Verify that all logging components encode data to prevent log injection. [CWE-117]	Fail	Escape newlines: log_msg.replace('\n','\n').replace('\r','\r')
7.4.1	1	Verify that a generic message is shown when an unexpected error occurs. [CWE-210]	Fail	@app.errorhandler(500): return jsonify(error='Internal error'), 500
7.4.2	2	Verify that exception handling is used across the codebase. [CWE-544]	Pass	Line 141: except signing.BadSignature:
7.4.3	2	Verify that a last resort error handler is defined. [CWE-460]	Fail	Register: @app.errorhandler(Exception) to catch all unhandled exceptions

Communication Security 1/6 (17%)

ID	L	Verification Requirement	Status	Finding / Remediation
9.1.1	1	Verify that TLS is used for all client connectivity. [CWE-319]	Pass	Line 559: "https://%s%s" % (host, request.get_full_path())
9.1.2	1	Verify that only strong cipher suites are enabled. [CWE-326]	Fail	Set nginx ssl_ciphers to ECDHE-RSA-AES256-GCM-SHA384 only
9.1.3	1	Verify that only the latest recommended versions of TLS are enabled, such as TLS 1.2 and TLS 1.3. [CWE-326]	Fail	Set: context.minimum_version = ssl.TLSVersion.TLSv1_2 in SSL context
9.2.1	2	Verify that connections to and from the server use trusted TLS certificates. [CWE-295]	Fail	Set verify=True in all requests.get/post — provide CA bundle path
9.2.3	2	Verify that all encrypted connections to external systems are authenticated. [CWE-319]	Fail	Remove verify=False from all requests calls — never disable cert check
9.2.4	2	Verify that proper certification revocation such as OCSP Stapling is enabled. [CWE-295]	Fail	Add to nginx: ssl_stapling on; ssl_stapling_verify on; resolver 8.8.8.8

Data Protection 0/7 (0%)

ID	L	Verification Requirement	Status	Finding / Remediation
8.1.1	2	Verify the application protects sensitive data from being cached in server components. [CWE-524]	Fail	Set Cache-Control: no-store on API responses containing sensitive data
8.2.1	1	Verify the application sets sufficient anti-caching headers. [CWE-312]	Fail	Add header: response.headers['Cache-Control'] = 'no-store, no-cache'
8.2.2	1	Verify that data stored in browser storage does not contain sensitive data. [CWE-312]	Fail	Move sensitive data from localStorage to httpOnly session cookie
8.2.3	1	Verify that authenticated data is cleared from client storage after session termination. [CWE-312]	Fail	On logout: localStorage.clear(); sessionStorage.clear(); clear DOM
8.3.1	1	Verify that sensitive data is sent to the server in the HTTP message body or headers. [CWE-359]	Fail	Move API keys from ?api_key=X to Authorization: Bearer X header
8.3.4	1	Verify that all sensitive data created by the application has been identified. [CWE-359]	Fail	Create data inventory document listing all PII fields and storage location
8.3.7	2	Verify that sensitive information is encrypted using approved algorithms. [CWE-359]	Fail	Encrypt PII columns: cipher.encrypt(value.encode()) before db.save()

API and Web Service 1/7 (14%)

ID	L	Verification Requirement	Status	Finding / Remediation
13.1.1	1	Verify that all application components use the same encodings and parsers. [CWE-116]	Fail	Use single JSON parser across all components — standardize on json module
13.1.3	1	Verify API URLs do not expose sensitive information such as the API key. [CWE-598]	Fail	Move ?api_key=X to Authorization: Bearer X in request headers
13.2.1	1	Verify that enabled RESTful HTTP methods are a valid choice for the user or action. [CWE-352]	Fail	Specify methods: @app.route('/api', methods=['GET','POST']) only
13.2.2	1	Verify that JSON schema validation is in place and verified before accepting input. [CWE-16]	Fail	Validate body: jsonschema.validate(request.json, REQUEST_SCHEMA)
13.2.3	1	Verify that RESTful web services are protected from Cross-Site Request Forgery. [CWE-352]	Pass	Line 629: REASON_CSRF_TOKEN_MISSING = "CSRF token missing."
13.2.5	2	Verify that REST services explicitly check the incoming Content-Type. [CWE-650]	Fail	Check: if 'application/json' not in request.content_type: abort(415)
13.4.1	2	Verify that query allow lists are used to prevent GraphQL DoS. [CWE-770]	Fail	Install graphql-depth-limit: depthLimit(5) and complexity limits

Configuration 3/11 (27%)

ID	L	Verification Requirement	Status	Finding / Remediation
14.2.1	1	Verify that all components are up to date. [CWE-1026]	Fail	Add to CI/CD: pip install safety && safety check -r requirements.txt
14.2.3	1	Verify that Subresource Integrity (SRI) is used for externally hosted assets. [CWE-829]	Fail	Add integrity attribute: <script src=cdn.js integrity=sha384-XXX crossorigin=ano
14.3.2	1	Verify that debug modes are disabled in production. [CWE-497]	Fail	Set in production.env: FLASK_DEBUG=0 FLASK_ENV=production
14.3.3	1	Verify that HTTP headers do not expose detailed version information. [CWE-200]	Fail	Add to nginx: server_tokens off; and remove X-Powered-By Flask header

14.4.3	1	Verify that a Content Security Policy (CSP) response header is in place. [CWE-693]	Fail	Add after_request: response.headers[CSP] = default-src self — in Flask after_req
14.4.4	1	Verify that all responses contain a X-Content-Type-Options: nosniff header. [CWE-693]	Pass	Line 543: self.content_type_nosniff = settings.SECURE_CONTENT_TYPE_NOSNIFF
14.4.5	1	Verify that a Strict-Transport-Security header is included on all responses. [CWE-693]	Pass	Line 540: self.sts_seconds = settings.SECURE_HSTS_SECONDS
14.4.6	1	Verify that a suitable Referrer-Policy header is included. [CWE-693]	Pass	Line 547: self.referrer_policy = settings.SECURE_REFERRER_POLICY
14.4.7	1	Verify that the content of a web application cannot be embedded in a third-party site. [CWE-116]	Fail	Add after_request: response.headers['X-Frame-Options'] = 'DENY'
14.5.1	1	Verify that the application server only accepts the HTTP methods in use. [CWE-346]	Fail	Specify methods=['GET','POST'] in route decorators — block all others
14.5.3	1	Verify that the CORS Access-Control-Allow-Origin header uses a strict allow list. [CWE-346]	Fail	Set CORS origins=['https://yourdomain.com'] — never origins=['*']

Files and Resources 0/6 (0%)

ID	L	Verification Requirement	Status	Finding / Remediation
12.1.1	1	Verify that the application will not accept large files that could cause denial of service. [CWE-400]	Fail	Add: app.config['MAX_CONTENT_LENGTH'] = 10 * 1024 * 1024 # 10MB limit
12.1.2	2	Verify that compressed files are checked against maximum allowed uncompressed size. [CWE-409]	Fail	Check: if len(zf.read(name)) > 100 * compressed_size: abort(400)
12.3.1	1	Verify that user-submitted filename metadata is not used directly to protect against path traversal. [CWE-22]	Fail	Sanitize filename: safe = os.path.basename(filename) before file ops
12.3.2	1	Verify that user-submitted filename metadata is validated to prevent LFI. [CWE-22]	Fail	Check filename: if not re.match(r'^[a-zA-Z0-9._-]+$', name): abort(400)
12.4.1	1	Verify that files from untrusted sources are stored outside the web root. [CWE-552]	Fail	Set UPLOAD_FOLDER='/var/uploads' — outside Flask static directory
12.5.1	1	Verify that the web tier is configured to serve only files with specific extensions. [CWE-552]	Fail	Configure nginx to serve only .html .css .js .png extensions